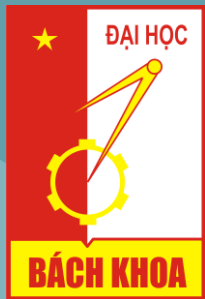


C PROGRAMMING INTRODUCTION



TUẦN 9: HÀM

**Trường Công nghệ Thông tin và Truyền thông
Trường Đại học Bách Khoa Hà Nội**

Hàm

```
double square(double a)
{
    return a * a;
}
```

- một nhóm các khai báo và câu lệnh được gán tên
 - thường trả về một giá trị
- một chương trình con
 - chương trình luôn có một hàm **main**
 - trong hàm **main** có thể gọi các hàm khác
 - các hàm này có thể gọi các hàm khác...

Bài thực hành 9.1. Bình phương

```
double square(double a)
{
    return a * a;
}
```

Hàm được định nghĩa
bên ngoài hàm main

```
int main(void)
{
```

```
    double num = 0.0, sqr = 0.0;
```

```
    printf("enter a number\n");
    scanf("%lf", &num);
```

```
    sqr = square(num);
```



Lời gọi hàm

```
    printf("square of %g is %g\n", num, sqr);
```

```
    return 0;
```

```
}
```

Tại sao sử dụng hàm?

- Chia nhỏ vấn đề ra các tác vụ con
 - dễ dàng xử lý các vấn đề phức tạp
- Tổng quát hóa một tập các câu lệnh lặp đi lặp lại
 - không phải gõ lại cùng một đoạn code nhiều lần
 - Ví dụ: hàm **printf** và **scanf**
- Chương trình dễ đọc và dễ bảo trì

Cú pháp của hàm

```
return-type name (argument-list)  
{  
    local-declarations;  
    statements;  
    return return-value;  
}
```

- Khi gọi hàm, có thể kèm theo **tham số của hàm**
- Khai báo tham số trong nguyên mẫu hàm

Bài thực hành 9.2

```
#include <stdio.h>
/* Ham in loi chao mung */

void sayHello ()
{
    printf("Hello World!\n");
}

/*
 * Goi ham in loi chao mung
 */

int main()
{
    sayHello();
    return 0;
}
```

Bài thực hành

9.3

```
#include <stdio.h>

int giaithua(int a) {
    int i, gt = 1;
    for(i=1; i<=a; i++)
        gt = gt * i;
    return gt;
}

int main() {
    int num;

    printf("Nhap so nguyen:");
    scanf("%d", &num);

    printf("%d! = %d\n",
           num, giaithua(num));
    return 0;
}
```

```
#include <stdio.h>

int giaithua(int) ;

int main() {
    int num;

    printf("Nhap so nguyen:");
    scanf("%d", &num);

    printf("%d! = %d\n",
           num, giaithua(num));
    return 0;
}

int giaithua (int a) {
    int i, gt = 1;
    for(i=1; i<=a; i++)
        gt = gt * i;
    return gt;
}
```

Bài thực hành 9.3

- a. Hãy thử nghiệm chương trình bên trái (hàm giaithua khai báo trước hàm main).
- b. Hãy thử nghiệm chương trình bên phải (hàm giaithua khai báo sau hàm main).
- c. Hãy tách chương trình thành các file như sau:
 - Hãy tách nguyên mẫu hàm giaithua sang file giaithua.h
 - Tách hàm giaithua sang file giaithua.c
 - Biên dịch và thử nghiệm lại chương trình.

Ví dụ: Project gồm 3 file hello.c, hello.h và week9_1.c

- File hello.h:

```
int binhphuong(int);
```

- File hello.c:

```
#include "hello.h"
```

```
int binhphuong(int a) {  
    return a * a;  
}
```

- Biên dịch:

```
gcc -o week9_1  
week9_1.c hello.c
```

- File week9_1.c

```
#include <stdio.h>
```

```
#include "hello.h"
```

```
int main() {  
    int n = 10;  
    printf("%d\n",  
           binhphuong(n));  
    return 0;  
}
```

Truyền tham số

- Tham số được truyền cho hàm bằng cách sao chép giá trị thay vì cung cấp truy cập đến biến
- Tham số thay đổi bên trong thân hàm sẽ không làm thay đổi giá trị của biến

Bài thực hành 9.4a

```
#include <stdio.h>

int addOne ( int i )
{
    i = i + 1;
    return i;
}

int main(void)
{
    int i = 3;

    printf("%d\n", addOne(i) );
    printf("%d\n", i);

    return 0;
}
```

- a. Mô tả kết quả khi chạy chương trình.
- b. Giải thích lý do thu được kết quả trên.

```
#include <stdio.h>
```

```
int i = 1;
```

```
int addOne ()
```

```
{
```

```
    i = i + 1;
```

```
    return i;
```

```
}
```

```
int main()
```

```
{
```

```
    printf("%d\n", addOne() );
```

```
    printf("%d\n", i);
```

```
    return 0;
```

```
}
```

Bài thực hành 9.4b Biến toàn cục

- Mô tả kết quả khi chạy chương trình.
- Giải thích lý do thu được kết quả trên.

```
#include <stdio.h>
```

```
int i = 1;
```

```
int addOne ()
```

```
{
```

```
    i = i + 1;
```

```
    return i;
```

```
}
```

```
int main()
```

```
{
```

```
    int i = 3;
```

```
    printf("%d\n", addOne() );
```

```
    printf("%d\n", i);
```

```
    return 0;
```

```
}
```

Bài thực hành 9.4c Biến toàn cục

- Mô tả kết quả khi chạy chương trình.
- Giải thích lý do thu được kết quả trên.

```
#include <stdio.h>

int i = 1;

int addOne (int i)
{
    i = i + 1;
    return i;
}

int main(void)
{
    int i = 3;

    printf("%d\n", addOne(i) );
    printf("%d\n", i);

    return 0;
}
```

Bài thực hành 9.4d

- a. Mô tả kết quả khi chạy chương trình.
- b. Giải thích lý do thu được kết quả trên.

Bài thực hành 9.5 - HUSTack

```
void badSwap ( int a, int b )
{ int temp;
  temp = a;
  a = b;
  b = temp;
  printf("Called environment: %d %d\n",a,b) ;
}

int main(void)
{ int a = 3, b = 5;
  printf("Calling environment: %d %d\n",a,b) ;
  badSwap( a, b );
  printf("Calling environment: %d %d\n",a,b) ;
  return 0;
}
```

- a. Mô tả kết quả khi chạy chương trình.
- b. Giải thích lý do thu được kết quả trên.
- c. Hãy sửa lại chương trình bằng cách sử dụng các biến a, b là các biến toàn cục.

Bài thực hành 9.6

```
int i;  
void f() {  
    int i = 0;  
    i++; // chỉ làm thay đổi giá trị biến i cục bộ  
}  
void g() {  
    i++; // làm thay đổi giá trị của biến i tổng thể  
}  
int main() {  
    i = 10;  
    f();  
    printf("i = %d", i);  
    g();  
    printf("\n i = %d", i);  
}
```

- Mô tả kết quả khi chạy chương trình.
- Tách các chương trình con f() và g() cùng với biến i (với từ khóa extern) vào file fg.c.

Bài thực hành 9.7 - HUSTack

- Viết hàm tính động năng của một vật
 $ke = mv^2/2$, với m là khối lượng (kg) và v là tốc độ (m/s)
`float kineticEnergy(float m, float v);`
- Hàm nhận vào hai tham số là m và v
- Viết và sử dụng hàm tính động năng này trong chương trình

Bài thực hành 9.8 - HUSTack

- Viết hàm `is_prime` nhận vào một số nguyên dương và trả về 1 nếu là số nguyên tố, 0 nếu ngược lại.

Nguyên mẫu:

```
int is_prime(int n);
```

- Viết chương trình cho người dùng nhập vào một số nguyên dương N và in ra màn hình tất cả các số nguyên tố từ 2 tới N sử dụng hàm trên

Bài thực hành 9.9

- Viết chương trình có các hàm sau và sử dụng chúng trong hàm main
 - Hàm tính tổng lập phương của các số từ 1 tới n: **long sumcube(int n) ;**
 - Hàm liệt kê các ước số của một số nguyên dương N: **void printsubmultiples(int n) ;**
 - Hàm liệt kê n bình phương của n số đầu tiên: **void printsquares(int n)**

Bài thực hành 9.10 - HUSTack

- Viết chương trình tính lương công nhân trong tuần.
- Lương trung bình là 150000 VND một giờ.
- Công nhân phải làm 40 giờ một tuần.
- Nếu làm thêm giờ, công nhân được trả gấp 1.5 số tiền trong giờ.
- Công nhân không được làm dưới 10 giờ hoặc hơn 65 giờ một tuần
- Yêu cầu sử dụng hàm với tham số đầu vào là số giờ làm trong tuần

long salary(int hours) ;

Bài thực hành 9.11

- Viết hàm

`void printnchars(int ch, int n)` để hiển thị một kí tự `n` lần.

Sử dụng hàm này để in ra tam giác * như sau

*

**

Bài thực hành 9.12

- Viết hàm chuyển đổi từ độ F sang độ C:

$$C = 5/9 (F-32)$$

`double celsius(double f);`

- In ra bảng liệt kê độ F từ 1 tới 20 và độ C tương ứng

Bài thực hành 9.13 - HUSTack

- Cho số nguyên dương N có k chữ số. Viết hàm kiểm tra xem tất cả k chữ số đều là số chẵn hoặc đều là số lẻ. Hàm trả về 1 nếu đúng và 0 nếu sai.

`int DigitAllSame(int n);`

- Viết chương trình sử dụng hàm trên để kiểm tra một số được người dùng nhập vào từ bàn phím.

Bài thực hành 9.14 - HUSTack

- Viết hàm *isLeapYear* nhận vào năm là tham số và kiểm tra có phải năm nhuận. Hàm trả về 1 nếu đúng và 0 nếu sai.

int isLeapYear(int year);

- Năm nhuận:
 - chia hết cho 4 nhưng không chia hết cho 100 OR
 - Nếu chia hết cho 100 thì phải chia hết cho 400
 - VD: 1996 và 2000 là năm nhuận, nhưng 1900, 2002 và 2100 không phải năm nhuận

Bài thực hành 9.15 - HUSTack

- Viết hàm xác định hàm $\sin(x)$ bằng cách dùng vòng lặp while hoặc do...while sử dụng khai triển Taylor với độ chính xác $1e-M$
 - Tham số: số thực x , số nguyên dương M
 - Giá trị trả về của hàm: $\sin(x)$
 - `float mySin(float x);`

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \dots + (-1)^{n-1} \frac{x^{2n-1}}{(2n-1)!} + o(x^{2n-1})$$

Bài thực hành 9.16 - HUSTack

- Viết hàm xác định hàm $\arcsin(x)$ bằng cách dùng vòng lặp while hoặc do...while sử dụng khai triển Taylor với độ chính xác $1e-M$
 - Tham số: số thực x , số nguyên dương M
 - Giá trị trả về của hàm: $\arcsin(x)$
 - float myArcsin(float x);
- Sử dụng hàm này để tính số PI với độ chính xác M (M được nhập từ bàn phím)
- Sử dụng hàm asin, hằng số PI của thư viện math.h
- So sánh với kết quả tính được ở phần trên.

$$\begin{aligned}\arcsin(z) &= z + \left(\frac{1}{2}\right) \frac{z^3}{3} + \left(\frac{1 \cdot 3}{2 \cdot 4}\right) \frac{z^5}{5} + \left(\frac{1 \cdot 3 \cdot 5}{2 \cdot 4 \cdot 6}\right) \frac{z^7}{7} + \cdots \\ &= \sum_{n=0}^{\infty} \frac{(2n-1)!!}{(2n)!!} \frac{z^{2n+1}}{2n+1} \\ &= \sum_{n=0}^{\infty} \frac{(2n)!}{(2^n n!)^2} \frac{z^{2n+1}}{2n+1}; \quad |z| \leq 1\end{aligned}$$

Bài thực hành 17

- Viết chương trình tính tổng sau:

$$1 + 1/2 + 1/3 + 1/4 + 1/5 + \dots + 1/n$$

float tinhTong(int n);

- Ví dụ:
 - Với $n = 5$
 - Kết quả: $1/1 + 1/2 + 1/3 + 1/4 + 1/5 = 2.283334$

Bài thực hành 9.18 - HUSTack

- Input: kích thước của diamond. Ví dụ: 5
- Viết hàm hiển thị hình thoi như sau (mỗi cạnh có chiều dài bằng 5).

```

    *
  * * *
* * * * *
* * * * * * *
* * * * * * * * *
  * * * * * * *
    * * * * *
      * * * *
        * * *
          *

```

`void printDiamond(int n);`

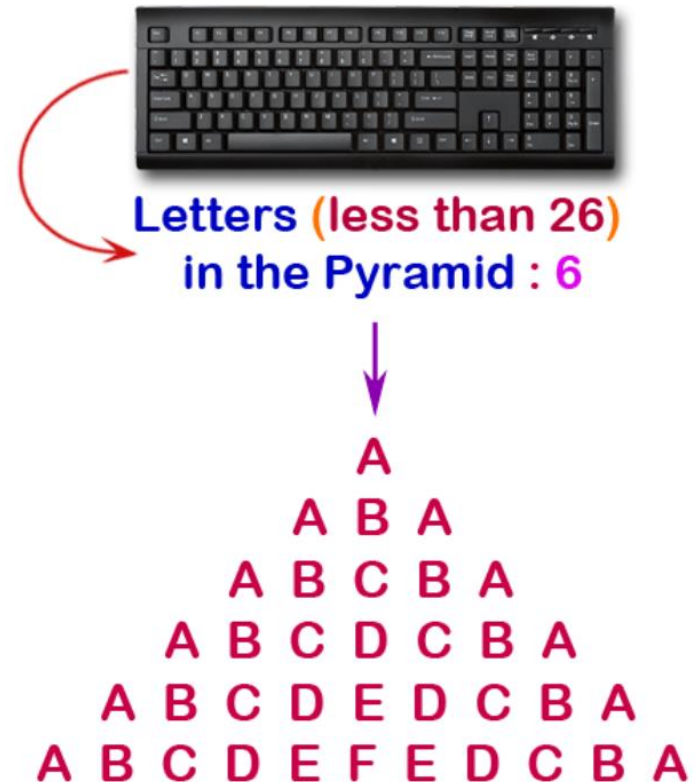
Bài thực hành 9.19 - HUSTack

- Viết hàm in ra hình sau

```
void printPyramid(int n);
```

- Input: số nguyên dương biểu diễn số dòng

```
      A
    A B A
  A B C B A
A B C D C B A
```



Bài thực hành 9.20 - HUSTack

- Viết hàm xác định ước số chung nhỏ nhất của hai số.

```
int highestCommonFactor (int a, int b);
```

- Ví dụ input:

24

28

Ví dụ *Output* :

4

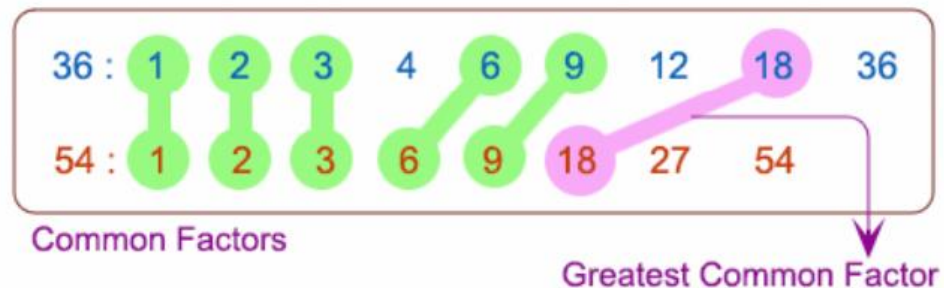
Determine the HCF of two numbers

List of Factors of 36 :

1 x 36, 2 x 18, 3 x 12, 4 x 9, 6 x 6

List of Factors of 54 :

1 x 54, 2 x 27, 3 x 18, 6 x 9



Bài thực hành 9.21 - HUSTack

- Viết hàm xác định bội số chung nhỏ nhất của hai số.

`int lowestCommonMultiple (int a, int b);`

- Ví dụ input:

15

20

- *Ví dụ Output :*

60

Bài thực hành 9.22

- Xác định hàm sinh(x) với độ chính xác $1e-10$
- `float mySinh(float x);`

$$\sinh x = x + \frac{x^3}{3!} + \frac{x^5}{5!} - \dots + \frac{x^{2n-1}}{(2n-1)!} + o\left(x^{2n-1}\right)$$

Bài thực hành 9.23

- Xác định hàm $\cosh(x)$ với độ chính xác $1e-10$
- `float myCosh(float x);`

$$\cosh x = 1 + \frac{x^2}{2!} + \frac{x^4}{4!} + \dots + \frac{x^{2n}}{(2n)!} + o(x^{2n})$$

Bài thực hành 9.24

- Xác định hàm $\arctan(x)$ với độ chính xác $1e-10$
- `float myArctan(float x);`

$$\arctan x = x - \frac{x^3}{3} + \frac{x^5}{5} - \dots + (-1)^{n-1} \frac{x^{2n-1}}{2n-1} + o\left(x^{2n-1}\right)$$

